

UE5 클라이언트 개발자 포트폴리오

RulesHorror / 마지막 특종

스토리형 호러 게임 프로토타입
플러그인 구조 · 3D 모니터 UI · StoryFlow 진행 시스템

장윤제 jjw1270@gmail.com

github.com/jjw1270/RulesHorror

먼저 보는 한 장 요약

프로젝트

마지막 특종

스토리형 호러 게임 프로토타입
UE5 / Windows / C++

내 작업 축

- 기능별 플러그인 분리
- 월드 컴퓨터·모니터 UX
- StoryFlow 진행 그래프
- DataTable·Registry 연결
- 저장 경계와 검증 도구

검토 포인트

3D ↔ 2D 입력 연결

모니터 표면 Hit를 UMG 입력으로 보정
해 전달

진행 런타임

Scene / Shot / Branch + 레벨 전이

데이터 기반 UI

StoryID → Registry → DataTable Row →
맵/사이트

제작 파이프라인

에디터 검증, 저장 실패 기준, 반복 테스트
경로

개발 범위와 현재 상태

시스템 구현 범위와 콘텐츠 보류 상태를 분리해서 정리했습니다.

구현 범위

상호작용

감지·타겟·피드백

모니터 UI

3D 입력·창 조작

진행 런타임

Scene·Shot·Branch

데이터

Registry·DataTable

저장 기반

Flow Ref 확장

현재 구현

- 인트로·새 게임·이어하기 진입 뼈대
- 상호작용 플러그인과 컴퓨터 Actor
- 모니터 입력 보정 + 멀티 윈도우 UI
- StoryFlow 그래프 런타임과 에디터
- 콘텐츠 데이터 조회와 SaveGame 확장 기반

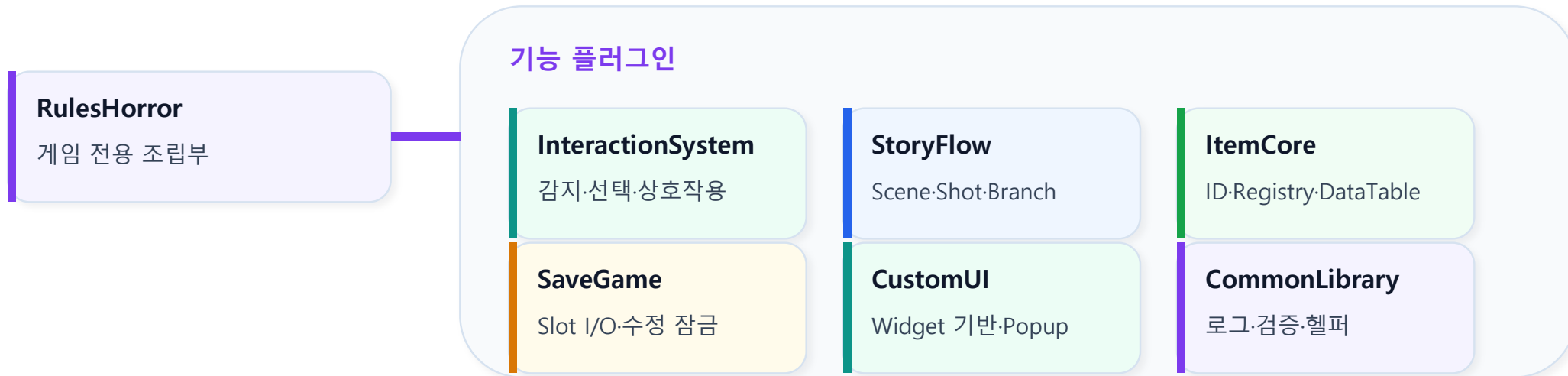
보류한 작업

- 본편 콘텐츠 제작과 연출 확장
- 플레이 구간별 아트 리소스 적용
- 진행 데이터와 UI 상태 확장

보류 사유: 게임 톤에 맞는 아트 리소스 수급 난이도

게임 본체는 조립부, 핵심 기능은 플러그인 경계로 분리

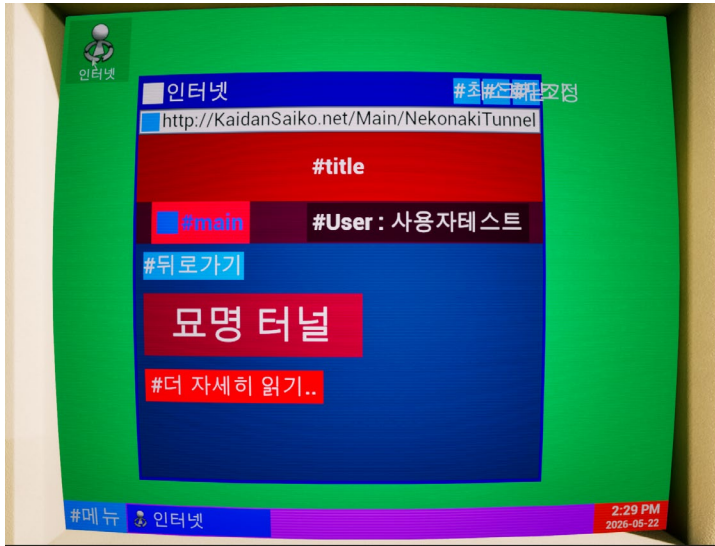
프로젝트 전용 흐름과 재사용 가능한 기반을 나눠 구현 범위를 읽기 쉽게 만들었습니다.



Content / BP / DataAsset / Map은 플러그인 기능을 실제 게임 흐름에 붙이는 프로젝트 문맥입니다.
RulesPlugin은 현재 실험 자리라 핵심 구현 범위에서 제외했습니다.

월드 안 컴퓨터를 조작하는 디제틱 UX

게임 외부 메뉴가 아니라, 월드 Actor와 모니터 UI가 연결된 상호작용 흐름으로 만들었습니다.



게임 내 모니터 화면

▶ [3D 모니터 조작 시연 영상](#)

상호작용 흐름



핵심 판단: 상호작용 즉시 UI를 띄우지 않고 '접근 → 도착 → 조작' 순서를 만들어 분위기와 입력 책임을 같이 잡았습니다.

컴퓨터 Actor와 Pawn이 역할을 나눠 진입을 완성

AComputer는 장치 상태를, OfficePawn은 입력 전달을 맡도록 나눴습니다.

Computer.cpp + OfficePawn.cpp

```
void AComputer::Interact_Implementation(AActor* Interactor)
{
    _OfficePawn = Cast<AOfficePawn>(Interactor);
    if (IsValid(_OfficePawn))
        return;

    _OfficePawn->SetTargetMovePoint(_InteractMovePoint, false,
        _OnMoveToPointFinishedEvent);
    _OfficePawn->SetInteractingComputer(this);
    SetPower(true, true);
}

void AOfficePawn::DriveWidgetInteraction(float DeltaTime)
{
    FHitResult Hit;
    const bool bHit = PC->GetHitResultUnderCursorByChannel(
        _MonitorScreenWidgetTraceType, true, Hit);
    BuildCorrectedMonitorWidgetHit(Hit);
    WidgetInteractionComponent->SetCustomHitResult(Hit);
}
```

코드가 보여주는 구현 판단

- AComputer는 기존 Interactable Interface에 편입
- MovePoint 도착 후 상호작용 전용 위젯 오픈
- WidgetComponent 화면을 Material에 연결
- Pawn 입력은 모니터 조작 중에만 활성화

책임 분리

장치 상태(AComputer)와 입력 경로 (OfficePawn)를 섞지 않아 이후 다른 월드 장치도 같은 상호작용 구조에 붙일 수 있습니다.

3D 모니터 표면 Hit를 2D UMG 입력으로 보정

이 구현의 핵심은 화면을 만드는 것보다, 월드 좌표와 위젯 좌표를 안정적으로 연결하는 데 있습니다.

입력 경로



World ImpactPoint

- WidgetComponent local Y/Z
- pivot 기준 UV
- 중심에서 멀수록 반경 기반 축 보정
- SetCustomHitResult

왜 직접 보정했나

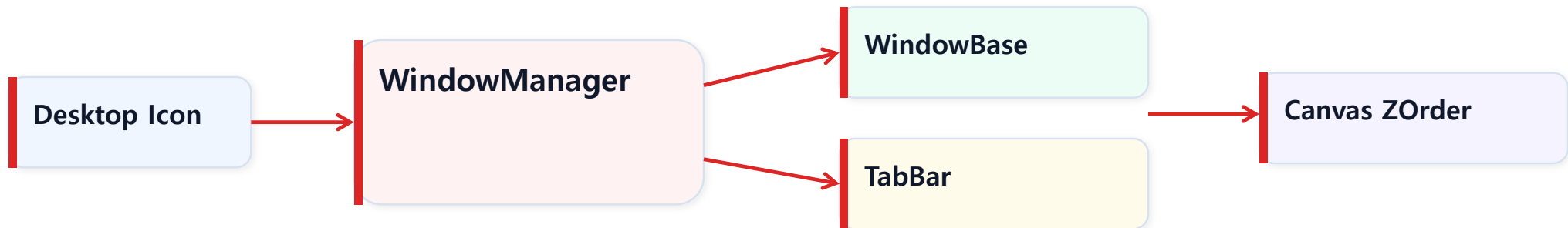
- 모니터 외곽에서 클릭 오차 체감
- hover / click / scroll 경로 통일 필요
- 실제 커서 피드백도 같은 Hit 기준 사용

문제 → 해결

**World Hit와 UMG 좌표계 차이를 숨기지 않고
입력 파이프라인으로 모델링했습니다.**

모니터 내부는 단일 팝업이 아니라 창 시스템

WindowManager가 창 상태, 탭, ZOrder를 함께 관리해 OS형 UX를 유지합니다.



구현 포인트

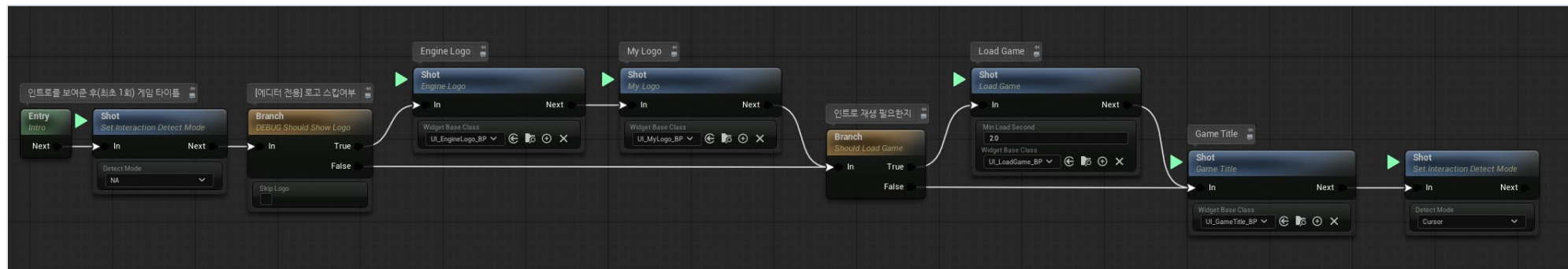
- 최소화·복구·닫기·포커스·리사이즈·최대화 상태를 공통 처리
- 탭 하이라이트와 최상단 창 계산을 중앙화
- 창 이동 중 최대화 해제까지 처리해 실제 데스크톱 감각을 보강

StoryFlow: Scene 내부 그래프로 진행을 편집

한 Scene 안에 Shot과 Branch를 배치하고, Transition 링크로 다음 Scene 전이를 표현했습니다.

Story Scene Editor 그래프

이미지의 노드들은 서로 다른 Scene이 아니라, 하나의 Scene 내부 진행 단위입니다.



Scene

장소/상황 단위 컨테이너입니다.
StorySceneAsset 하나가 Scene 하나를 나타내며,
SceneID·TargetLevel·SceneTemplate과 내부 Shot/Branch 목록을 보유합니다.

Shot

실제 진행을 수행하는 최소 실행 단계입니다.
UStoryShotBase 템플릿을 런타임 인스턴스로 만들고
Enter/Tick/Finish/Exit 이후 NextLink로 다음 대상으로 이동합니다.

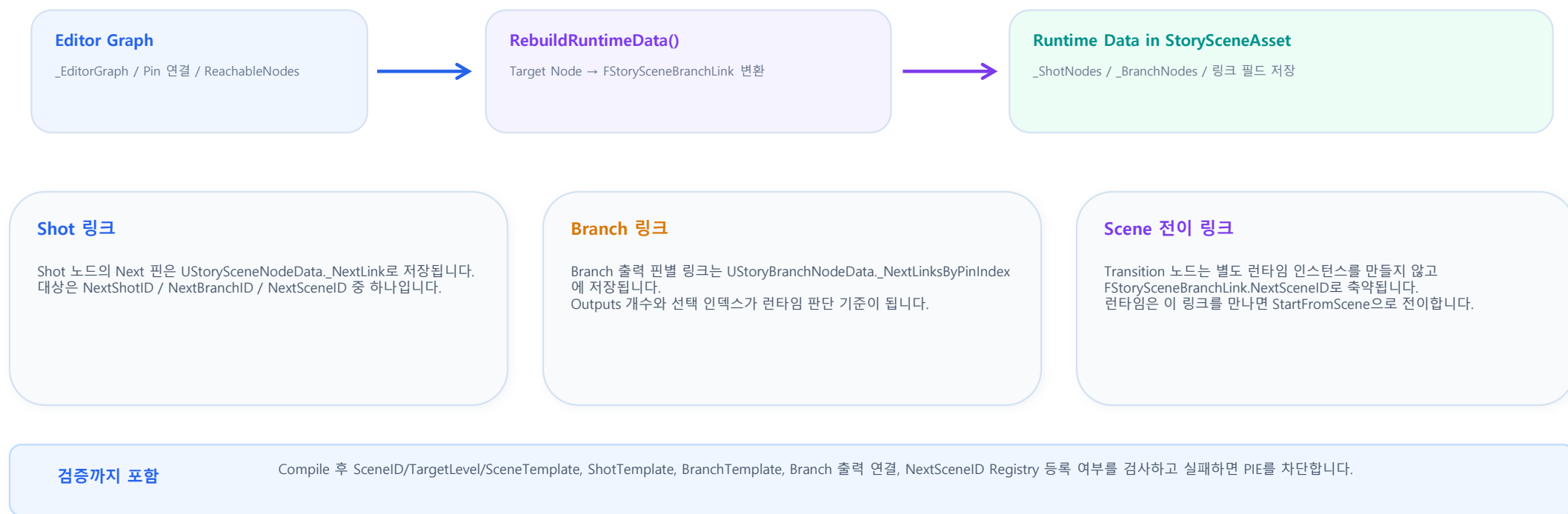
Branch

조건에 따라 다음 경로를 고르는 판단 단계입니다.
UStoryBranchBase의 Outputs와 SelectNextIndex 결과로 여러 링크 중 하나를 선택합니다. 저장 기준은 아닙니다.

StoryFlow: 그래프를 런타임 데이터로 Compile

StorySceneEdGraph::RebuildRuntimeData 기준으로, 에디터 노드 연결을 실행 가능한 링크 데이터로 변환했습니다.

코드 기준 핵심: 그래프 배치는 런타임이 직접 순회하지 않고, Compile 단계에서 SceneAsset 내부 링크 데이터로 재작성됩니다.

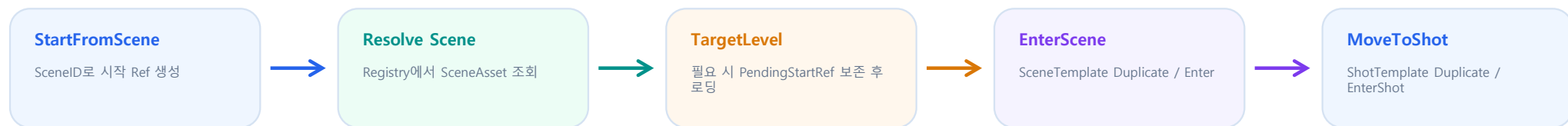


근거: StorySceneEdGraph.cpp, StorySceneEditor.cpp

StoryFlow: 런타임은 Shot / Branch 링크만 소비

StoryFlowSubsystem 기준으로 Scene 시작, Shot 실행, Branch 판단, Scene 전이가 하나의 링크 처리 루프로 이어집니다.

런타임 진입 경로는 **StartFromScene** → **StartFromRef** → **StartResolvedScene**으로 수렴하고, 이후 **MoveToLink**가 Shot / Branch / Scene 링크를 분기 처리합니다.



Shot 실행 루프

MoveToShot은 ShotNode의 ShotTemplate을 DuplicateObject로 인스턴스화합니다.
FinishCurrentShot 호출 시 ExitShot 후 GetNextLink를 읽고 MoveToLink로 넘깁니다.

Branch 판단 루프

MoveToLink가 Branch 링크를 만나면 EvaluateBranch를 호출합니다.
BranchTemplate Outputs 수를 확인하고 SelectNextIndex 결과를 clamp한 뒤 해당 핀 링크로 이동합니다.

Scene 전이 / 복원

MoveToLink가 Scene 링크를 만나면 StartFromScene(NextSceneID)로 재진입합니다.
현재 위치는 GetCurrentRef가 SceneID + ShotID로 만들며, Branch는 저장 지점이 아닙니다.

설계 효과

에디터 그래프, 레벨 전환, 런타임 실행, 저장/복원을 분리해 검증 가능한 데이터 경계와 재개 지점을 확보했습니다.

근거: StoryFlowSubsystem.cpp, StorySceneAsset.h, StorySceneNodeData.h

레벨 이동 중에도 진행 참조를 잃지 않게 분리

LoadingLevel과 TargetLevel 전이를 분리하고 PendingStartRef로 시작 위치를 보존했습니다.

StoryFlowSubsystem.cpp

```
bool UStoryFlowSubsystem::StartFromRef(const FStoryFlowRef& Ref)
{
    UStorySceneAsset* SceneAsset = FindSceneAssetBySceneID(Ref.SceneID);
    if (IsValid(SceneAsset))
        return false;

    if (ShouldOpenTargetLevel(SceneAsset))
        return BeginPendingSceneTravel(SceneAsset, Ref);

    return StartSceneAsset(SceneAsset, Ref.ShotID);
}

// Loading level and target level are separated.
// PendingStartRef preserves SceneID + ShotID during travel.
```

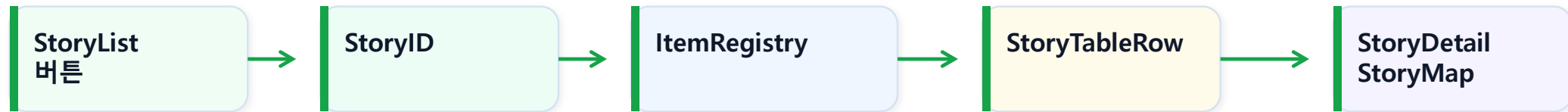
제작 파이프라인까지 포함

- Scene Registry로 SceneID resolve
- Shot/Branch 링크 Compile
- TargetLevel·Branch 출력 무결성 검증
- Shot 노드 단독 PIE 테스트 경로 제공

진행 시스템을 블루프린트 연결이 아니라 런타임·에디터 검증·반복 테스트 비용 문제로 다뤘습니다.

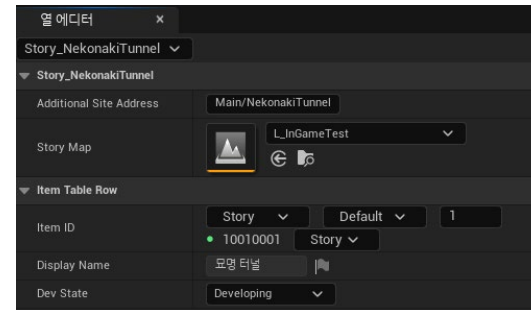
사이트 UI는 맵을 직접 알지 않고 StoryID를 조회

StoryList와 StoryDetail이 DataTable/Registry 경계 위에서 콘텐츠를 표시합니다.



데이터 연결 의도

- 콘텐츠 항목 추가 시 UI 코드보다 Row 추가를 우선
- 목록·상세·맵 진입을 StoryID 하나로 연결
- 프로젝트 Row는 ItemCore 기본 Row를 확장



Story DataTable Row

데이터 오류는 런타임 전에 Registry 단계에서 드러냄

ItemID 검증과 RowStruct 호환성 검사를 콘텐츠 제작 경계에 배치했습니다.

ItemRegistrySubsystem.cpp

```
bool UItemRegistrySubsystem::IndexItemTable(const UDataTable* Table)
{
    const FItemID& ItemID = ItemRow->ItemID;
    FItemIDValidationResult Result = ItemID.Validate();
    if (Result.IsInvalid() || Contains(ItemID))
        return false;

    _ItemTypeIndexMap.FindOrAdd(ItemID.GetType())
        .ItemIDToRow.Add(ItemID, MoveTemp(RowReference));
    return true;
}

// StoryDetail resolves StoryID -> StoryTableRow -> StoryMap.
```

검증 대상

- invalid ItemID
- 중복 ID
- 지원하지 않는 RowStruct
- Registry DataAsset 로드 실패

효과

데이터가 늘어날수록 추적이 비싸지는 오류를 Editor Message Log와 Registry Refresh 흐름에서 먼저 발견합니다.

저장은 플러그인 경계와 게임 확장 지점을 분리

공통 SaveGame 안정성 정책 위에 RulesHorror 전용 StoryFlowRef를 얹었습니다.

SaveGameSubsystem.cpp

```
void USaveGameSubsystem::AsyncSaveGame()
{
    if (_IsAsyncSaving || _IsAsyncLoading)
        return;

    _IsAsyncSaving = true;
    SetSaveGameCanModify(false);
    UGameplayStatics::AsyncSaveGameToSlot(_SaveGame, Slot, 0,
        [this](..., bool bSuccess)
        {
            _IsAsyncSaving = false;
            SetSaveGameCanModify(true);
        });
}
```

현재와 다음

현재

단일 Slot I/O
async 수정 잠금
StoryFlowRef 저장 기반

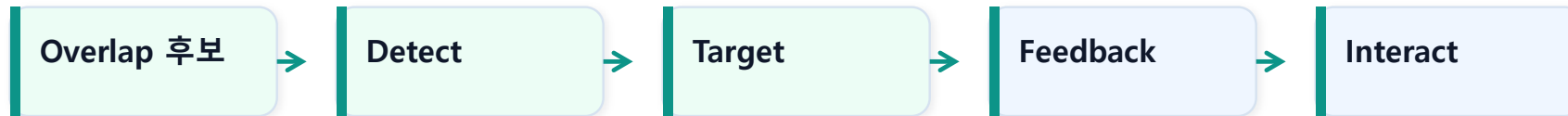
확장

프로젝트별 필드 확장
저장 UI 피드백 연동
복원 상태 범위 확대

핵심 판단: Load 실패 시 기존 메모리 객체를 유지하고, 비동기 중 수정 가능 여부를 별도 정책으로 둡니다.

상호작용 대상 차이는 Interface 뒤로 숨김

감지·선택·피드백·Interact 호출은 UInteractorComponent가 공통 처리합니다.



Component가 관리

- CameraCenter / Cursor 감지 모드
- None / Detected / Targeted 상태
- Indicator / Overlay 동기화

Actor가 구현

- CanBeDetected / CanInteract
- InteractionLocation / Overlay Mesh
- 실제 Interact 동작

문제 해결 사례 1: 대표 사례

3D 입력 좌표 보정과 Soft Reference 패키징 누락 대응

3D 모니터 입력 보정

월드 Hit가 맞아도 클릭 위치가 밀림

- 증상** 모니터 외곽에서 커서 피드백과 실제 UMG 입력 체감이 어긋남
- 원인** 3D 표면 충돌 지점과 Widget 좌표계가 1:1로 맞지 않고 외곽 오차가 커짐
- 구현** 로컬 좌표 → UV → 반경 기반 축 보정 후 SetCustomHitResult로 hover.click.wheel 경로 통일

남은 판단: 모니터 Mesh나 DrawSize가 바뀌면 보정 강도는 다시 튜닝

World Hit



UV 보정



Custom Hit

Primary Data Asset 기반 Cook 추적

Soft Reference만 둔 DataTable이 패키징에서 누락됨

- 증상** 에디터에서는 Item/Story DataTable 조회가 되지만 패키징 빌드에서 Registry 로드 실패
- 원인** DeveloperSettings의 TSoftObjectPtr은 런타임 로드 경로일 뿐 Cook 포함을 보장하지 않음
- 구현** UItemRegistryDataAsset을 UPrimaryDataAsset으로 두고 GetPrimaryAssetId(ItemRegistry, Name)로 AssetManager 추적 기준 제공

남은 판단: AssetManager 스캔/Cook Rule은 프로젝트 설정과 패키징 로그로 계속 검증

Soft Ref



Primary DataAsset



Cook 검증

문제 해결 사례 II: 보조 사례

진행 복원, 창 상태, 저장 안정성, 데이터 검증 리스크를 작은 단위로 정리

레벨 전이 참조 보존

로딩을 거치면 시작 shot이 흔들림

증상 TargetLevel 이동 뒤 어느 진행 단위부터 재개할지 책임이 섞이기 쉬움

원인 레벨 travel과 시작 참조가 결합되면 로딩 경계에서 상태 복원이 불안정

구현 FStoryFlowRef(SceneID+ShotID)를 저장 단위로 두고 PendingStartRef를 분리해 TargetLevel 진입 후 Shot 시작

남은 판단: Branch는 순간 판단으로 두고 확정 Shot 기준으로만 복원

Loading



StartRef



Target Shot

창 포커스/ZOrder 증앙화

창이 겹칠 때 입력 대상과 탭 상태가 어긋남

증상 최상단 창, 탭 하이라이트, 최소화/복구 상태가 서로 다르게 보일 수 있음

원인 각 Window가 포커스와 ZOrder를 직접 판단하면 상태 동기화 지점이 분산 됨

구현 WindowManager가 활성 창, 탭, ZOrder 계산을 중앙화하고 WindowBase는 상태 반영에 집중

남은 판단: 창 타입이 늘어나면 공통 정책과 예외 정책을 분리 유지

Focus



ZOrder



Tab UI

추가 사례

저장 안정성

문제: 저장 중 수정 시점 불명확
조치: async 동안 수정 잠금, Load 실패 시 기존 객체 유지
효과: 진행 상태 유실로 전파되지 않게 방어

데이터 검증

문제: DataTable 증가 시 ID 오류 추적 비용 증가
조치: RowStruct-invalid-duplicate를 Registry Refresh에서 검사
효과: 런타임 전 Editor Message Log로 발견

배치 파일로 동기화부터 실행까지 반복 비용 절감

최신 소스를 받은 뒤 프로젝트 파일 생성·빌드·에디터 실행을 Windows 배치로 묶었습니다.

BuildAndLaunch.bat

원클릭 실행 경로

Git

> Subtree

> Submodule

> ProjectFiles

> Editor Build

Build.bat 호출 뒤 에디터 실행까지 한 흐름으로 연결

.uproject 실행

스크립트 역할 분리

SyncBranch.bat

fetch/pull + Subtree + Submodule

GenerateProjectFiles.bat

UE 엔진 경로 검증
Visual Studio 파일 생성

작은 진입점 3개로
동기화 / 생성 / 전체 실행을 나눴습니다.

자동화 안의 안전장치

- Git 충돌 상태면 pull을 밀어붙이지 않음
- UE_EngineDir.txt 캐시 후 경로 유효성 재검증
- .uproject 1개 기준을 확인해 잘못된 빌드 대상 방지
- 단계별 errorlevel 실패 시 로그와 함께 종료

이 포트폴리오에서 봐주셨으면 하는 역량

게임 화면뿐 아니라 기능을 나누고, 연결하고, 검증 가능한 형태로 만든 과정을 담았습니다.

시스템 분해

Plugin · Subsystem · Interface 경계

UE 런타임 연결

World Actor ↔ Widget ↔ Flow ↔
Save

제작 파이프라인

Editor 검증 · 반복 테스트 · 문서화

감사합니다.